Quantitative Portfolio Optimization

Team NAPPERS

Summary

Goal: Develop an end-to-end portfolio optimization pipeline that constructs monthly, long-only equity portfolios on an S&P 500–like universe and evaluates them against simple benchmarks using strictly out-of-sample performance.

Current Project Scope:

- Focuses on daily, long-only, fully-invested equity portfolios; excludes leverage, shorting, options, and intraday trading.
- Restricts evaluation to historical backtests on the daily sample.
- Includes infrastructure, testing, and documentation as primary goals so that future extensions (e.g. new signals, risk models, or universes) can plug into the same framework.

After observing near-equal-weight behavior, targeted verification experiments were conducted to confirm the optimizer and constraints are implemented correctly—upon further verification it appeared that the optimiser was implemented correctly but rebalancing penalties were initially too steep.

Objective

$$\max_{w_t} \quad \mu_t^\top w_t - \gamma\, w_t^\top \Sigma_t w_t - \lambda \|w_t\|_1 - \kappa \|w_t - w_{t-1}\|_1, \quad \text{s.t.} \ \sum_i w_{t,i} = 1, \ w_{t,i} \geq 0,$$

This objective balances four components:

Expected return: rewards allocations toward assets with higher estimated mean returns (estimated from rolling historical data).

Risk penalty: penalizes portfolio variance, with γ\gammaγ controlling risk aversion.

Weight regularization: discourages extreme or highly concentrated allocations by penalizing large absolute weights.

Turnover penalty: penalizes deviations from last month's portfolio, serving as a proxy for transaction costs and encouraging stability over time.

Constraints enforced via softmax parameterization in PyTorch to guarantee long-only, fully-invested portfolios.

Data

Sourced from Wharton Research Data Services (WRDS)

S&P 500 daily data (2000 - present)

Useful fields include: date, ticker, price, return, shares outstanding, SIC codes

Pros:

- Dataset is cleaned and reputable
- Long historical coverage (25+ years of daily data, over 3,000,000 rows)
- Breadth of data (returns are pre-calculated, outstanding shares and volume included)

Cons:

- Limited in scope beyond straightforward applications of portfolio optimization
- Huge dataset - computationally expensive, could lead to overfitting
- Access limitation - potentially leads to complications with theoretical distribution/real-world implementation of model
-

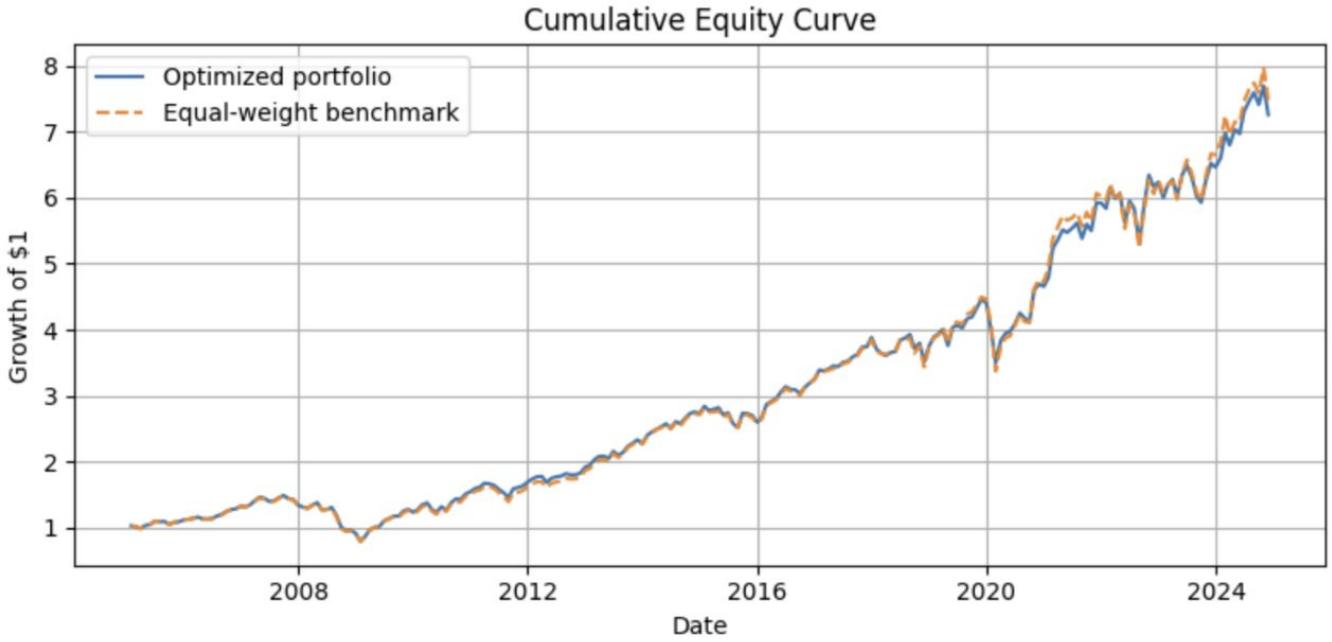
Moving forward:

- As project advances, we may need to acquire supplementary data - finer or coarser grain date-wise depending on eta
- In order to differentiate from previous project, may need to search for more niche dataset to narrow our application (ESG, alternate optimization strategy such as a two layer model or higher moments)
- If pivoting to prediction markets, data reputability and cleanliness decreases, risks accuracy or hallucination since sourced from GAI

Baseline Results

- Baseline hyperparameters:
  - $\gamma = 5.0$
  - $\kappa = 0.1$
  - learning rate $= 0.1$
  - steps $= 200$
- Optimized portfolio:
  - mean monthly return: 0.0092
  - annualized volatility: 0.1415
  - annualized Sharpe: 0.7774
  - cumulative return: 6.2591
- Equal-weight benchmark remains very close throughout.

Baseline Equity Curve



Baseline Equity Curve

- The optimized portfolio and equal-weight benchmark are nearly indistinguishable.
- The baseline specification remains close to an equal-weight regime.

Why So Close to Equal Weight?

- Three possible explanations:
  - the objective or constraints are implemented incorrectly,
  - the optimizer is not moving far from initialization,
  - the baseline specification is simply too conservative.
- In particular:
  - return estimates may be weak,
  - the risk penalty may dominate,
  - the turnover penalty may anchor the portfolio to past weights.
- This motivates a set of sanity checks.

Two-Asset Sanity Check

- Select one "good" stock and one "bad" stock from the full return panel.
- Criterion:
  - high versus low average realized monthly return,
  - at least one full rolling window of overlap.
- Selected pair:
  - good stock mean return: 0.0542
  - bad stock mean return: -0.0271
  - overlap: 34 months
- Goal: test whether the optimizer tilts toward the better asset rather than defaulting to 50–50.

Two-Asset Sanity Check Result

- Estimated mean returns over the last 24 overlapping months:

$$\mu_t = \begin{bmatrix} 0.0472 \\ -0.0022 \end{bmatrix}$$

- With $\kappa = 0$, the optimizer chooses:

$$w_t = \begin{bmatrix} 0.6089 \\ 0.3911 \end{bmatrix}$$

- This is already a clear deviation from equal weight.
- Conclusion: the optimizer is functioning correctly and is not mechanically producing 50–50 portfolios.

Convergence Check

- Relax penalties further:
  - $\gamma = 1.0$
  - $\kappa = 0$
- Increase optimization aggressiveness:
  - learning rate $= 2.0$
  - steps $= 2000$
- Result:

$$w_t = \begin{bmatrix} 0.9943 \\ 0.0057 \end{bmatrix}$$

- The optimizer can move far from equal weight when the signal is strong and the search is allowed to converge.

Implication of the Sanity Checks

- The baseline similarity to equal weight is not due to a coding error.
- The objective and constraints are implemented correctly.
- The optimizer can find strongly tilted portfolios in simple settings.
- Therefore, the baseline near-equal-weight behavior reflects:
  - conservative penalty choices,
  - limited optimizer movement,
  - weak cross-sectional signal in the full problem.

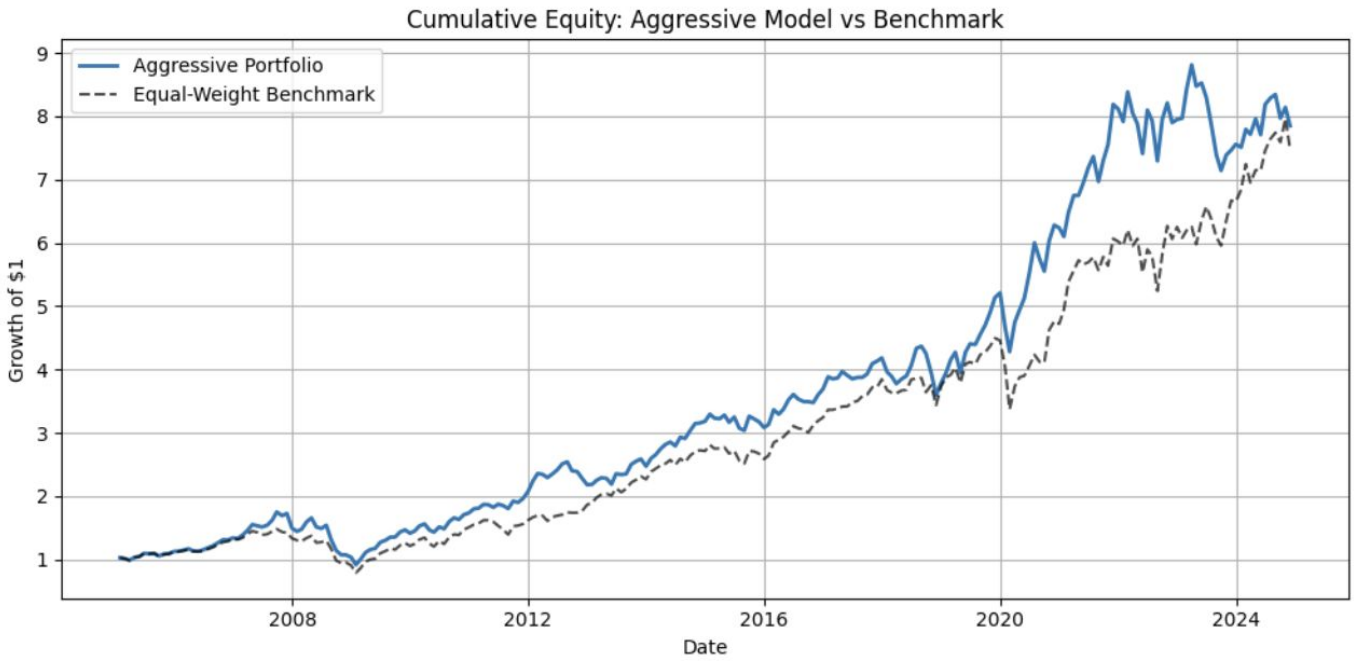
Aggressive Full-Sample Specification

- Re-run the full backtest with weaker turnover control.
- New specification:
  - $\gamma = 5.0$
  - $\kappa = 0.005$
  - learning rate $= 0.1$
  - steps $= 1000$
- Same data, same dynamic universe, same benchmark.
- Purpose: test whether the full strategy can move out of the near-equal-weight regime.

Aggressive Results

- Aggressive portfolio:
  - mean monthly return: 0.0096
  - annualized volatility: 0.1514
  - annualized Sharpe: 0.7626
  - cumulative return: 6.8492
- Equal-weight benchmark:
  - mean monthly return: 0.0094
  - annualized volatility: 0.1476
  - annualized Sharpe: 0.7620
  - cumulative return: 6.4788
- Higher cumulative return, but with slightly higher risk.

Aggressive Equity Curve



Aggressive Equity Curve

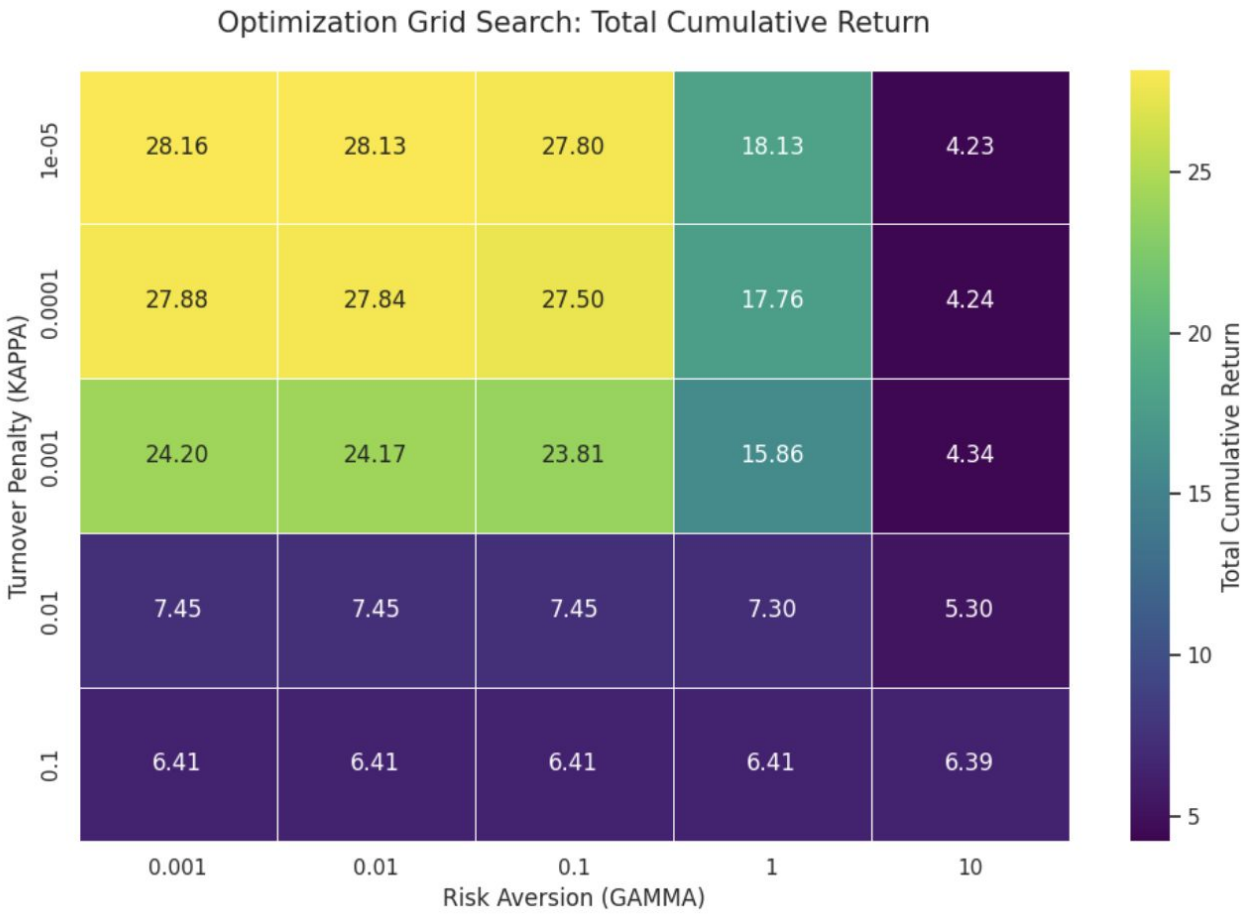
- The optimized portfolio now separates more clearly from equal weight.
- The gain is mainly in cumulative return rather than Sharpe ratio.

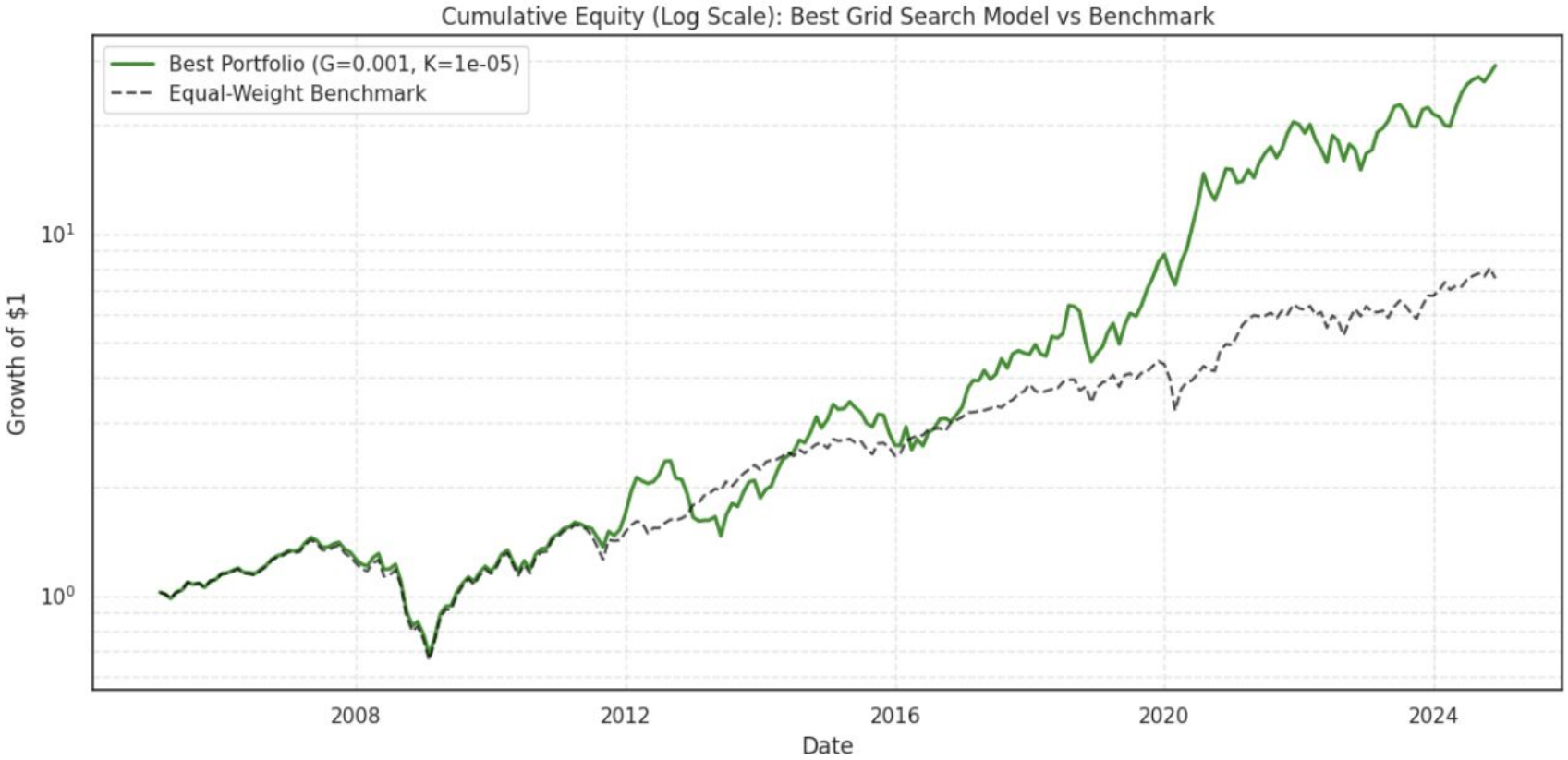
Portfolio Diagnostics

- Average L^1 distance from month-specific equal weight: 0.9049
- Average monthly turnover: 0.0289
- Under the baseline specification, turnover was only 0.0052.
- Hence the aggressive specification is genuinely more active:
  - larger deviations from equal weight,
  - more frequent reallocation through time.

Next Step: Hyperparameter Search

- The results are sensitive to the choice of hyperparameters.
- The next step is to search systematically over:
  - risk aversion γ,
  - turnover penalty κ,
  - learning rate,
  - number of optimization steps,
  - rolling window length.
- Objective:
  - identify which parameters drive deviation from equal weight,
  - and whether any specification improves out-of-sample performance robustly.


Optimization Grid Search: Total Cumulative Return
Turnover Penalty (KAPPA)
1e-05
0.0001
0.001
0.01
0.1
28.16
28.13
27.80
18.13
4.23
27.88
27.84
27.50
17.76
4.24
24.20
24.17
23.81
15.86
4.34
7.45
7.45
7.45
7.30
5.30
6.41
6.41
6.41
6.41
6.39
0.001
0.01
0.1
1
10
Risk Aversion (GAMMA)
Total Cumulative Return
25
20
15
10
5


Cumulative Equity (Log Scale): Best Grid Search Model vs Benchmark
Best Portfolio (G=0.001, K=1e-05)
Equal-Weight Benchmark
Growth of $1
10^1
10^0
2008
2012
2016
2020
2024
Date

Exploration: Skew Tensor

Let $X \in \mathbb{R}^n$ be a random vector with mean μ.

Define the third-order central moment tensor (coskewness tensor):

$$\mathcal{S}_{ijk} = \mathbb{E}\left[(X_i - \mu_i)(X_j - \mu_j)(X_k - \mu_k)\right]$$

$$\mathcal{S}_{ijk} = \mathcal{S}_{jik} = \mathcal{S}_{ikj} = \mathcal{S}_{kij} = \mathcal{S}_{kji} = \mathcal{S}_{jik}.$$

Case	**Indices**	**Representative Entry**	**Number of Unique Terms**	**Growth**
Single	$i = j = k$	$\mathcal{S}_{iii} = \mathbb{E}[(X_i - \mu_i)^3]$	n	$\mathcal{O}(n)$
Double	$i = j \neq k$	$\mathcal{S}_{iik} = \mathbb{E}[(X_i - \mu_i)^2(X_k - \mu_k)]$	$n(n-1)$	$\mathcal{O}(n^2)$
Triple	$i \neq j \neq k$	$\mathcal{S}_{ijk} = \mathbb{E}[(X_i - \mu_i)(X_j - \mu_j)(X_k - \mu_k)]$	$\binom{n}{3} = \frac{n(n-1)(n-2)}{6}$	$\mathcal{O}(n^3)$
Total	—	—	$\frac{n(n+1)(n+2)}{6}$	$\mathcal{O}(n^3)$

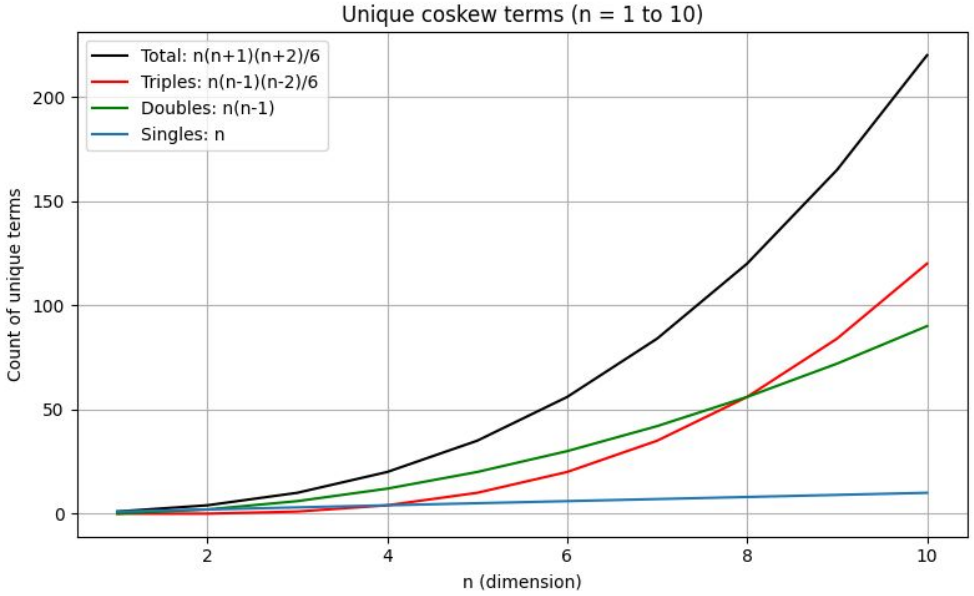
Exploration: Skew Tensor 2

As dimensionality increases, the structure of coskewness becomes overwhelmingly dominated by

triple-variable interactions.

This has implications for:

- Storage complexity,
- Estimation stability,
- Regularization needs,
- Dimensionality reduction strategies.



Exploration: Skew Tensor 3

For portfolio optimization we want to maximize the portfolio skew at the cost of portfolio variance, where portfolio skew is defined by the contraction of the tensor over the weight indices.

$$\boxed{\text{Skew}_p = \sum_{i,j,k} w_i w_j w_k \mathcal{S}_{ijk}}$$

$$\text{Skew}_p = \mathcal{S}[w, w, w].$$

$$\text{Skew}_p = w^\top \mathcal{S}(w, w),$$

Formulation	Objective	First-Order Condition (FOC)
Skew–Variance Tradeoff	$\max_w \mathcal{S}[w,w,w] - \lambda w^\top \Sigma w \quad \text{s.t. } \mathbf{1}^\top w = 1$	$3\,\mathcal{S}(w,w) - 2\lambda\Sigma w - \gamma\mathbf{1} = 0$
Unconstrained Skew–Variance	$\max_w \mathcal{S}[w,w,w] - \lambda w^\top \Sigma w$	$3\,\mathcal{S}(w,w) = 2\lambda\Sigma w$
Scale-Invariant (Skew Ratio)	$\max_w \frac{\mathcal{S}[w,w,w]}{(w^\top \Sigma w)^{3/2}}$	$\mathcal{S}(w,w) = \theta\,\Sigma w$
Mean–Variance–Skew	$\max_w \mu^\top w - \frac{\lambda}{2} w^\top \Sigma w + \eta\mathcal{S}[w,w,w] \quad \text{s.t. } \mathbf{1}^\top w = 1$	$\mu - \lambda\Sigma w + 3\eta\,\mathcal{S}(w,w) - \gamma\mathbf{1} = 0$

Ideas for GPU Usage

- Monte Carlo Simulation for optimization
  - Simulate thousands of return scenarios in parallel using GPU tensor operations
  - Replace slow loops with vectorized PyTorch computations (CUDA-enabled)
  - Enable more robust portfolio optimization under uncertainty (scenario-based)
  - Improve runtime → allows deeper sensitivity analysis and parameter sweeps
  - Resampling the efficient frontier by averaging optimal portfolio weights across simulations
-
- GPU-based hyperparameter sweeps
  - Run large-scale parameter sweeps (risk aversion, constraints, signal weights)
  - Compare multiple portfolio construction strategies in parallel
  - Leverage GPU to speed up model training and evaluation loops
  - Identify robust configurations across market scenarios
- RAPIDS library cuDF/cuML for large panel data
  - Accelerate large panel data operations (joins, groupbys) on GPU
  - Handle firm × time datasets more efficiently than traditional pandas
  - Reduce preprocessing bottlenecks in feature engineering pipeline
  - Most impactful for high-frequency or very large datasets

Pivot to Prediction Markets

- Original approach (traditional portfolio optimization) lacked novelty relative to prior projects
- Prediction markets offer a distinct asset class with different structure and payoff dynamics than traditional markets
- Contracts directly encode market-implied probabilities of real-world events
- Allows integration of statistical modeling and market efficiency testing
- Aligns with modern quant strategies (information aggregation, mispricing detection)
- Space to make real, interesting progress and discoveries

Prediction Markets Literature Review

- Prediction markets trade contracts that pay $1 if an event occurs → prices ≈ probabilities
- Proven to be effective forecasting tools, often outperforming polls and experts
- Markets aggregate dispersed information
- Not fully efficient
- Biases (overconfidence, underreaction)
- Liquidity constraints
- Ample arbitrage opportunity

Implication**:** discrepancies between market probabilities and model predictions may create exploitable opportunities

Literature Reviewed:

- Justin Wolfers & Eric Zitzewitz (2004): *Prediction Markets*
- Robin Hanson (2003): *Combinatorial Information Market Design*
- Philip Tetlock (2015): *Superforecasting*

Prediction Markets Data

Used API endpoint:

- `gamma-api.polymarket.com/markets`

Parsed JSON using Python `(json)`
Flattened nested data using `pandas.json_normalize`
Handled nested fields (lists/dictionaries) for CSV export

- Dataset: `polymarket_markets_rich.csv`
- Contains:
  - Market metadata (question, category, status)
  - Pricing / probability fields
  - Liquidity / volume-related fields
- Structured for downstream analysis

Next Steps Data-Wise

- Data expansion
- Pull historical price data (time series) for selected markets
- Extract key fields:
  - `market_id`, `timestamp`, `price`, `volume`
- Build a panel dataset across markets and time

Align data into consistent format:

- one row per `(market_id, time)`

Handle:

- missing timestamps
- irregular update intervals

Create derived features:

- returns / price changes
- volatility measures
- time-to-resolution

Betting Markets Problem Formulation

Problem: Construct a dynamic portfolio of Polymarket contracts that maximizes expected edge relative to market-implied probabilities, net of spread, slippage, and turnover, while controlling for bankroll risk, event concentration, and liquidity constraints.

Data: Use Polymarket market metadata, price history, order-book snapshots, open interest, volume, and related-event structure available through the Polymarket CLI.

Core idea: Replace noisy equity expected-return estimates with model-implied probabilities on binary contracts, and replace generic covariance with event-structure-aware scenario risk.

Extensions: consistency arbitrage, Kelly sizing, and short-horizon alpha.

Let:

- $i = 1, \ldots, N_t$ index active markets at time t
- $c_{i,t} \in [0,1]$ be the current executable contract price
- $\hat{p}_{i,t} \in [0,1]$ be your model-estimated fair probability
- $w_{i,t}$ be portfolio capital weight in contract i
- w_{t-1} be previous holdings
- τ_i be the market resolution time

Define the **edge** on each contract as

$$e_{i,t} = \hat{p}_{i,t} - c_{i,t}^{exec}$$

where $c_{i,t}^{exec}$ should be the actual executable price

$$\max_{w_t} \sum_i w_{i,t}(\hat{p}_{i,t} - c_{i,t}^{exec}) - \gamma \sum_i w_{i,t}^2 \hat{p}_{i,t}(1 - \hat{p}_{i,t}) - \kappa \|w_t - w_{t-1}\|_1$$

subject to

$$w_i \geq 0, \quad \sum_i w_i \leq 1, \quad w_i \leq u_i, \quad \sum_{i \in \mathcal{E}_j} w_i \leq U_j$$

Probability Estimation and Optimization

Stage 1: Estimate fair probabilities

Train a model to produce $\hat{p}_{i,t}$.

Start simple:

- logistic regression,
- gradient-boosted trees,
- or regularized online logistic regression.

Useful features:

- current best bid / ask / midpoint,
- spread,
- order-book imbalance,
- recent return over several horizons,
- realized volatility,
- volume / open interest,
- time to expiry,
- category / tag / event embeddings,
- related market prices within the same event.

$$\hat{p}_{i,t} = f_\theta(x_{i,t})$$

Stage 2: Optimize weights

Projected-gradient algorithm

Initialize with previous weights:

$$w^{(0)} \leftarrow w_{t-1}$$

For iteration $m = 0, \ldots, M-1$:

1. Compute gradient of

$$J(w) = e_t^\top w - \gamma w^\top \Sigma_t w - \lambda \|w\|_2^2$$

as

$$\nabla J(w) = e_t - 2\gamma \Sigma_t w - 2\lambda w$$

2. Take a gradient step:

$$\tilde{w}^{(m+1)} = w^{(m)} + \eta \nabla J(w^{(m)})$$

3. Handle turnover penalty approximately with soft-thresholding or proximal update:

$$\tilde{w}^{(m+1)} \leftarrow \mathrm{prox}_{\eta\kappa\|\cdot - w_{t-1}\|_1}(\tilde{w}^{(m+1)})$$

4. Project onto feasible set:

$$w^{(m+1)} = \Pi_{\mathcal{C}}(\tilde{w}^{(m+1)})$$

where $\mathcal{C}$ is:

- $w_i \geq 0$,
- $\sum_i w_i \leq 1$,
- per-market caps,
- per-event caps.

Return $w_t = w^{(M)}$.

Exploration - Polymarket

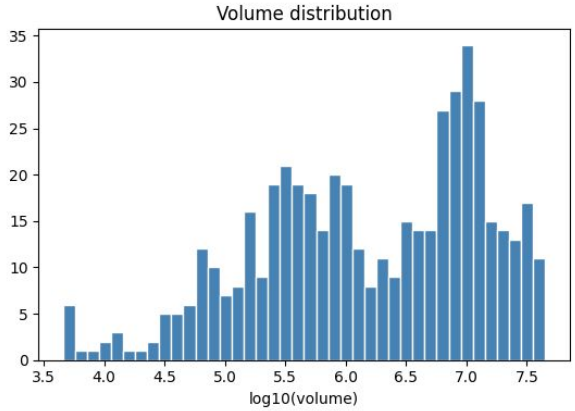
Snapshot contains about **500 markets and 92 columns**

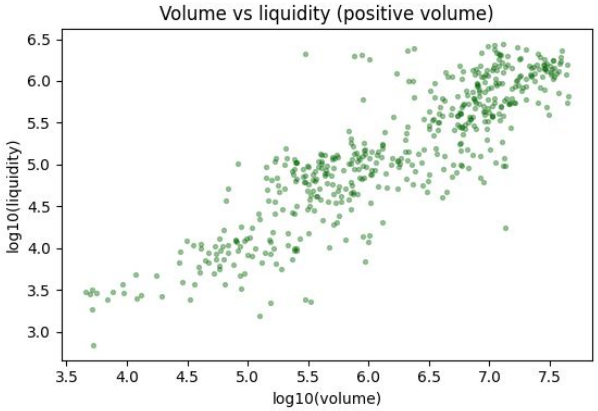
Key fields used:

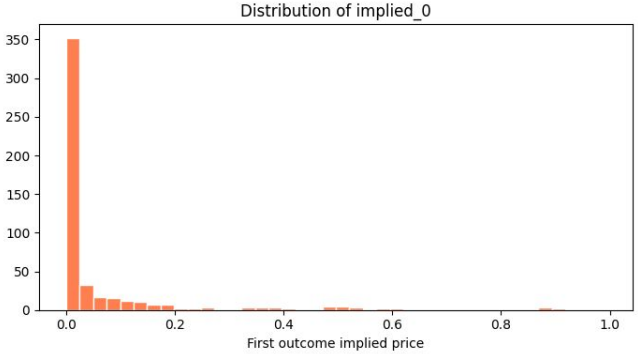
- question / market metadata
- outcome prices
- best bid / best ask / spread
- volume and liquidity
- event structure

Derived fields include:

- **implied probability (first list price)**
- **event ID / event title** for grouping and concentration control







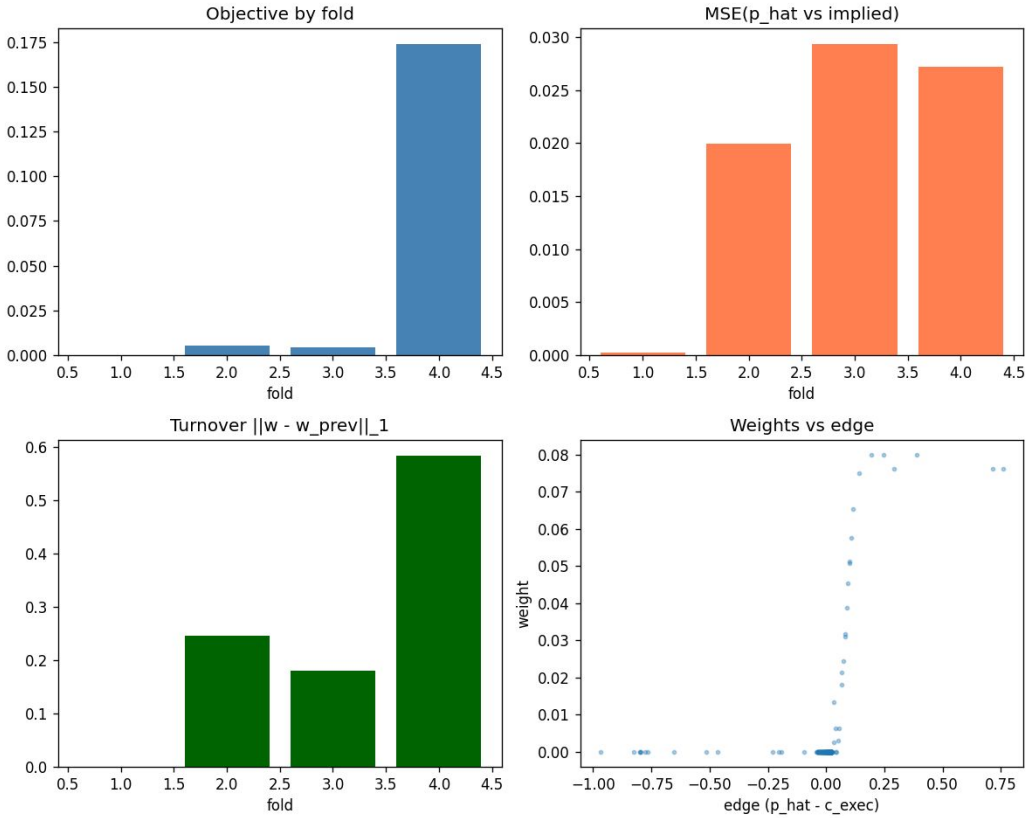
Implementation

Baseline results

- **Mean objective:** 0.0459
- **Mean MSE between model probability and implied probability:** 0.0192
- **Mean turnover:** 0.2527
- **PnL:** -2.77%, +2.37%, -0.58% across folds 2–4)

Interpretation:

- positive objective suggests the optimizer is finding some estimated edge
- model differs meaningfully from market prices
- turnover is moderate, so trading frictions still matter
- performance appears somewhat **fold-dependent**, so robustness is not yet established
- simulated PnL was inconsistent across folds



Implementation Architecture

Files/modules built

- `src/polymarket/load.py` — load and enrich snapshot
- `src/polymarket/features.py` — feature engineering
- `src/polymarket/execution.py` — executable price and liquidity logic
- `src/polymarket/model_baseline.py` — baseline p^\hat pp^ model
- `src/polymarket/optimizer.py` — projected-gradient optimizer
- `src/polymarket/backtest.py` — walk-forward evaluation
- `src/polymarket/diagnostics.py` — charts
- `scripts/run_polymarket_mispricing.py` — main run script

Outputs generated

- portfolio weights
- trades
- fold-level PnL
- diagnostics plots
- metrics by fold
- summary JSON / TXT reports

Continuation

Key limitations

- Current dataset is a **single cross-sectional snapshot**
- Markets are open; there are **no realized resolutions in-file**
- True out-of-sample realized PnL is limited without a historical panel of snapshots or resolution data
- Current evaluation is best viewed as a **research baseline**, not proof of deployable alpha

Next extensions

- **Consistency arbitrage** across related markets
- **Kelly sizing** for bankroll-aware position sizing
- **Short-horizon alpha** for forecasting near-term price moves
- richer execution model with fees, ticks, and deeper order-book effects